HealthNexus Auth and Onboarding Guide

How people sign up, sign in, stay signed in, recover their password and set up their profile, and the files behind each step.

This is the companion to the **HealthNexus Feature Guide**, which covers every other feature. Same layout: what the user sees, the files that build it, the steps, and the rules worth knowing. Backend paths start at `backend/app/` and frontend paths at `frontend/src/`.

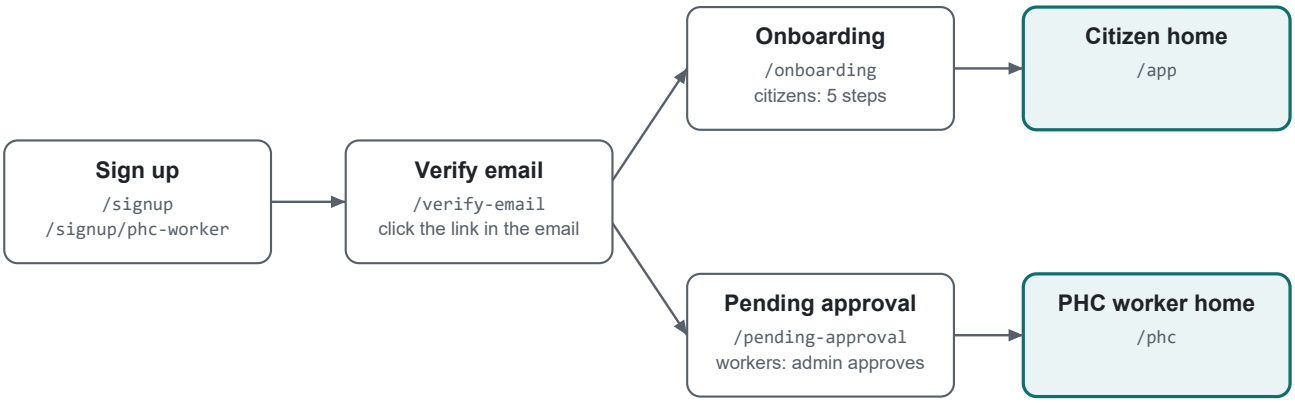
Last updated: 8 October 2026

Contents

- 1. The journey of a new account, and the four account states
- 2. Auth: signup (citizen and PHC worker), email verification, login, staying signed in, logout, devices, forgot and reset password, page guards
- 3. Onboarding: the 5-step profile setup
- 4. Security decisions, explained simply
- 5. Tests, settings and the cheat sheet

1. The journey of a new account

Every account goes through the same gates. Citizens verify their email, then fill in a 5-step profile. PHC workers verify their email, then wait for an administrator to approve them. Only then do they reach the app.



Account states

Two fields on the `users` table decide where a person can go: `role` and `is_verified`. Citizens also have `profile_completed`. The single function `destinationFor()` in `lib/routes.ts` turns these into a page, and login, the page guards and every Continue button use it.

State	What it means	Where they land
Not verified	Signed up, but has not clicked the email link yet.	`/verify-email`
Citizen, no profile	role = `user`, verified, profile_completed = false.	`/onboarding`
Citizen, ready	role = `user`, verified, profile done.	`/app`
Pending PHC worker	role = `pending_phc`: signed up as a worker, not approved yet.	`/pending-approval`
PHC worker	role = `phc_worker`: approved.	`/phc`
Admin	role = `admin`.	`/admin`

Who approves PHC workers? An administrator. There is no approval screen or API endpoint yet (the admin console is "Coming soon"). The demo worker is approved by the seed script `seeds/seed_phc_worker.py`, which also links the worker to a health centre using `services/phc_registry.py` and writes a `phc_verification_audit_log` entry.

2. Auth

All auth endpoints live in one file, `api/routes/auth.py`, under `/api/v1/auth`. The rules behind them are in `services/accounts.py` (accounts, emails, passwords) and `services/sessions.py` (signed-in sessions). On the frontend, `context/AuthContext.tsx` holds the signed-in user for the whole app.

2.1. Sign up

Two sign-up forms: one for citizens and one for PHC workers. A worker also gives their employee ID, job title, health centre name, district, state and years of service.

Side	File	What it does
Frontend	`pages/SignupPage.tsx`	Citizen sign-up page.
Frontend	`pages/PHCSignup.tsx`	PHC worker sign-up page.
Frontend	`components/auth/SignupForm.tsx`	One form for both. The "phc" variant adds the worker fields and an example employee ID.
Frontend	`lib/validation.ts, lib/patterns.ts`	The same rules as the backend, checked when you leave a field.
Frontend	`services/auth.ts`	registerCitizen() and registerPhcWorker().
Backend	`api/routes/auth.py`	POST /auth/register/citizen and POST /auth/register/phc-worker.
Backend	`services/accounts.py`	_create(): makes the user (and the worker's application) in one transaction.
Backend	`schemas/auth.py`	Checks the input: email format, password length, employee ID pattern.
Backend	`core/security.py`	hash_password() with bcrypt.
Backend	`services/mailer.py`	Sends the verification email in the background.
Database	`users, phc_applications`	Tables this feature reads or writes.

What happens, step by step

1. The form checks the fields, then sends them.
2. The backend refuses an email that is already used (409).
3. It creates the user with a **hashed** password. A citizen gets role `user`; a worker gets role `pending_phc` and a row in `phc_applications` with status pending.
4. It creates a verification token. The raw token goes into the email link; only its hash is saved.
5. The email is sent after the response, so sign-up stays fast. The user goes to the "check your inbox" screen.

Rules worth knowing (useful when someone asks)

- Password: at least 8 characters and at most 72 bytes. bcrypt only reads 72 bytes, so longer passwords are refused instead of being silently cut.
- Employee ID must look like TNPHC-CHN-2847. The pattern is in core/patterns.py, and the frontend keeps an exact copy; a test fails if they differ.
- Job titles: medical officer, staff nurse, health inspector, ASHA supervisor, block health supervisor.
- If two sign-ups with the same email arrive at the same moment, the database's unique rule catches the second one.

2.2. Email verification

A link in the email confirms the address. Until then the person can only see the "Verify your email" screen and their device list.

Side	File	What it does
Frontend	`pages/VerifyEmail.tsx`	With a token in the URL it confirms it; without one it is the "check your inbox" screen with a resend button.
Frontend	`services/auth.ts`	verifyEmail() and sendVerification().
Backend	`api/routes/auth.py`	POST /auth/email/verify (public) and POST /auth/email/send-verification (signed in).
Backend	`services/accounts.py`	redeem_verification() and prepare_verification_email().
Backend	`services/mailer.py`	Real email when SMTP_HOST is set; otherwise prints the email and link to the console for development.
Database	`users`	Tables this feature reads or writes.

What happens, step by step

1. The person clicks the link. The page sends the token to /auth/email/verify.
2. The backend hashes the token and looks for a user with that hash. If it is missing or older than 24 hours, the link is refused.
3. It sets is_verified and removes the token, so the link works only once.
4. Resend: a new link replaces the old one. Asking again within 60 seconds is refused with a wait time (429).

Rules worth knowing (useful when someone asks)

- The verify page is public on purpose: people often open the link on a phone with no session.
- Logs never contain a full email address or a link (mask_email()).

2.3. Log in

Email and password. On success the person goes straight to the right page for their state (section 1).

Side	File	What it does
Frontend	`pages/LoginPage.tsx`, `components/auth/LoginForm.tsx`	The form. Also shows why a session ended (for example "signed out due to inactivity").
Frontend	`context/AuthContext.tsx`	login(): saves the user and marks the app as signed in.
Frontend	`services/auth.ts`	login().
Backend	`api/routes/auth.py`	POST /auth/login.
Backend	`services/accounts.py`	authenticate(): checks the password.
Backend	`services/sessions.py`	start_session(): creates the refresh token and the device record.
Backend	`core/security.py`	create_access_token(): the 15-minute JWT.
Database	`users`, `refresh_tokens`	Tables this feature reads or writes.

What happens, step by step

1. The backend finds the user by email and checks the password with bcrypt.
2. It creates a session row in refresh_tokens (hash of the token, expiry, device such as "Chrome on Windows", IP address).

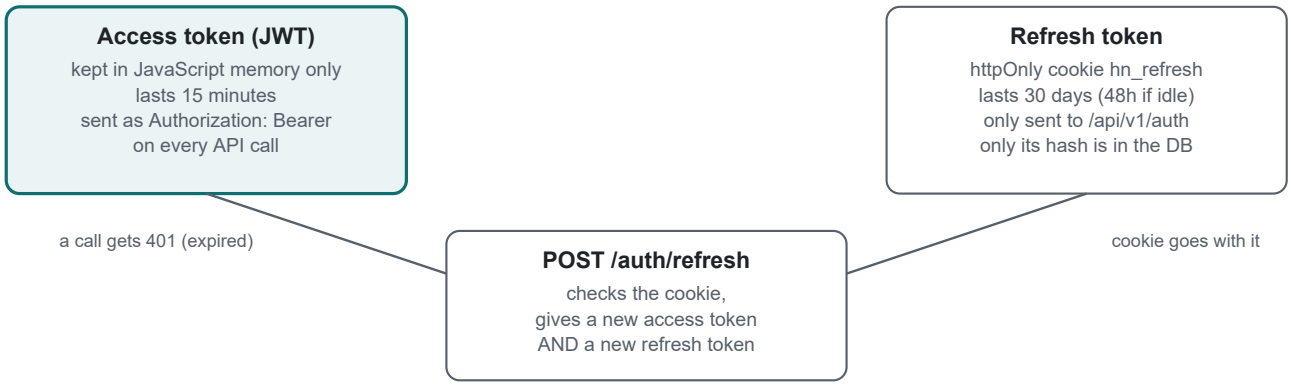
3. The response sets the refresh token as an httpOnly cookie and returns the access token plus the user's details.
4. The frontend keeps the access token in memory and sends the person to destinationFor(user).

Rules worth knowing (useful when someone asks)

- One message, "Incorrect email or password", for a wrong password, an unknown email and a disabled account, so nobody can find out which emails have accounts.
- For an unknown email the backend still runs a fake password check (burn_password_check), so the response takes the same time.

2.4. Staying signed in (tokens explained)

The app uses two tokens. The short one is used for every request; the long one is only used to get a new short one.



Side	File	What it does
Frontend	`services/api.ts`	Adds the access token to every call. When a call gets 401, refreshes once and repeats the call. Only one refresh runs at a time.
Frontend	`context/AuthContext.tsx`	On page load, restores the session from the cookie (restoreSession). When a session ends, clears the user and saved onboarding answers.
Backend	`api/routes/auth.py`	POST /auth/refresh and GET /auth/me.
Backend	`services/sessions.py`	rotate_session(): checks the cookie and swaps it for a new one.
Backend	`core/dependencies.py`	get_current_user(): reads the access token, then loads the user fresh from the database on every request.

What happens, step by step

1. Any API call sends the access token. After 15 minutes it expires and the server answers 401.
2. api.ts calls POST /auth/refresh. The browser sends the hn_refresh cookie automatically.
3. The backend checks the token's hash: not revoked, not expired (30 days), and used within the last 48 hours.
4. **Rotation:** the old refresh token is marked used and a new one is issued (linked to the old one). The response sets the new cookie and returns a new access token.
5. api.ts repeats the original call with the new token. The person notices nothing.

Rules worth knowing (useful when someone asks)

- **Why two tokens?** The access token is short-lived and only in memory, so if it is stolen it is useless after 15 minutes. The refresh token is in an httpOnly cookie, which JavaScript cannot read, so a script injected into the page cannot steal it.

- **Theft detection:** each refresh token works once. If an already-used token comes back after more than 10 seconds, someone copied it, so every newer token in that chain is revoked and both copies are signed out.
- **Two tabs at once:** if two tabs refresh within 10 seconds, the second gets "already used" but its session is left alone. The frontend waits 400 ms and tries again with the new cookie.
- **Role changes take effect at once:** the user's role and active status are read from the database on every request, not from the token, so an approval or a deactivation applies immediately.
- The JWT algorithm is fixed to HS256 in code and the secret must be at least 32 characters, or the server refuses to start (main.py).

2.5. Log out, and signed-in devices

Sign out of this device, or see every device that is signed in and end any of them.

Side	File	What it does
Frontend	`context/AuthContext.tsx`	logout(): signs out locally even if the request fails, and clears saved onboarding answers.
Frontend	`pages/account/SessionsPage.tsx`	The list of devices, with "this device" marked, and buttons to end one or all others.
Frontend	`services/auth.ts`	logout(), listSessions(), revokeSession(), revokeOtherSessions().
Backend	`api/routes/auth.py`	POST /auth/logout, GET /auth/sessions, DELETE /auth/sessions/{id}, POST /auth/sessions/revoke-others.
Backend	`services/sessions.py`	revoke_by_token(), revoke_all_for_user(), active_sessions(), current_session_id(), device_label().
Database	`refresh_tokens`	Tables this feature reads or writes.

What happens, step by step

1. Logout marks the refresh token as revoked (reason "logout") and deletes the cookie.
2. The device list shows live sessions only: not revoked, not expired, used in the last 48 hours.
3. "Sign out other devices" revokes everything except the current session. If the server cannot tell which one is current, it refuses rather than signing you out too.

Rules worth knowing (useful when someone asks)

- Every revoked session keeps a reason: rotated, logout, inactivity, reuse_detected, revoked_by_user, password_reset.
- Device names are kept simple ("Chrome on Windows"), enough to recognise a device without tracking it.

2.6. Forgot and reset password

Enter your email to get a reset link, then choose a new password. Every device is signed out.

Side	File	What it does
Frontend	`pages/ForgotPassword.tsx`	Asks for the email; always shows the same "if an account exists" message.
Frontend	`pages/ResetPassword.tsx`	Opened from the link; asks for the new password.
Frontend	`services/auth.ts`	forgotPassword() and resetPassword().
Backend	`api/routes/auth.py`	POST /auth/password/forgot and POST /auth/password/reset.
Backend	`services/accounts.py`	prepare_password_reset() and redeem_password_reset().

Side	File	What it does
Database	`users, refresh_tokens`	Tables this feature reads or writes.

What happens, step by step

1. Forgot: if the email belongs to an active account, a reset token (valid for 1 hour) is saved as a hash and emailed.
2. The answer is the same whether or not the account exists.
3. Reset: the backend checks the token, saves the new hashed password, removes the token, and revokes all sessions (reason "password_reset").

Rules worth knowing (useful when someone asks)

- The same answer for every email means nobody can use this page to find out who has an account.
- Asking again within 60 seconds sends nothing, so the page cannot be used to spam someone's inbox.
- Signing out every device matters: if someone else knew the old password, they are thrown out too.

2.7. Page guards (who can open which page)

Pages check the person's state before showing anything, so a page never appears for someone the API would refuse.

Side	File	What it does
Frontend	`components/ProtectedRoute.tsx`	Signed out goes to /login. Wrong role goes to /unauthorized. Not verified goes to /verify-email. A citizen with no profile goes to /onboarding.
Frontend	`components/RedirectIfAuthenticated.tsx`	Landing, login and signup are only for signed-out visitors; signed-in users are sent to their home.
Frontend	`pages/PendingApproval.tsx`	What a pending worker sees: their application details and that an administrator must approve it.
Frontend	`pages/errors/Unauthorized.tsx, NotFound.tsx`	Error pages.
Frontend	`main.tsx`	The route table: each page wrapped in the right guard and allowed roles.
Backend	`core/dependencies.py`	The backend guards: get_current_user, require_user, require_phc_worker, require_admin, require_onboarding_user, require_any_verified.

What happens, step by step

1. The frontend guard decides which page to show; the backend guard decides what data to return. Both use the same rules, but **the backend is the one that counts**: hiding a page is for convenience, refusing the data is the security.

Rules worth knowing (useful when someone asks)

- require_onboarding_user = a citizen with a verified email.
- require_any_verified = any approved role with a verified email. A pending worker is refused ("Your account is not active yet").
- Identity and role always come from the token and the database, never from what the request body says.

3. Onboarding

After verifying their email, a citizen fills in a 5-step profile. The most important step is choosing their **health centre**: it decides which PHC workers will see their readings (the catchment).

Step	What it asks	Rules
1. About you	Age, gender, height, weight, blood group (optional).	Age 1 to 120, height 50 to 300 cm, weight 1 to 300 kg.
2. Location	State, then district, then health centre; pincode and area.	Each list loads from the choice before it. The centre must be active and in that district.
3. Medical history	Existing conditions and current medicines, as tags.	Tags are optional; duplicates are removed.
4. Device and contact	Smartwatch (and brand), emergency contact name and phone.	A brand needs "has smartwatch"; a contact name needs a phone and the other way round.
5. Consent	Health tracking? Community surveillance?	Both must be answered yes or no; there is no default.

3.1. The onboarding flow

A step-by-step form with a progress indicator. Answers survive a page reload, and the final submit saves everything at once.

Side	File	What it does
Frontend	`pages/Onboarding.tsx`	The flow: the 5 steps, checking each step before moving on, and the final submit. A citizen who already has a profile is sent away.
Frontend	`components/onboarding/steps.tsx`	One component per step: PersonalStep, LocationStep, MedicalStep, DeviceStep, ConsentStep.
Frontend	`lib/onboardingDraft.ts`	Saves answers in sessionStorage so a reload does not lose them. Cleared when the tab closes or on sign-out.
Frontend	`lib/useForm.ts`	Form state: checks a field when you leave it, and puts server errors back onto the right field.
Frontend	`components/ui/TagInput, Select, Field`	The inputs used by the steps.
Frontend	`services/onboarding.ts`	getStates(), getDistricts(), getFacilities(), createProfile().
Backend	`api/routes/onboarding.py`	GET /onboarding/states, /districts, /facilities; POST, GET and PATCH /onboarding/profile. Every route needs a verified citizen.
Backend	`services/onboarding.py`	Registry lookups, resolve_facility() (checks the chosen centre), apply_fields() (copies values onto the tables).
Backend	`schemas/onboarding.py`	ProfileFields: every field's limits and the cross-field rules.
Backend	`services/consent.py`	record_consents(): writes the two consent answers.
Database	`users, user_profiles, consent_records, valid_phc_facilities`	Tables this feature reads or writes.

What happens, step by step

1 The location step loads the states that have an active health centre, then that state's districts, then its centres. Each centre shows how many workers serve it.

2. On submit, the backend checks the chosen centre is active and really is in that state and district.
3. In **one transaction** it creates `user_profiles`, links the citizen to the centre (`users.phc_id`), sets profile_completed, saves the health-tracking choice, and writes both consent answers with source "onboarding".
4. The frontend refreshes the user (refreshUser), clears the draft and goes to /app.

Rules worth knowing (useful when someone asks)

- Only districts with at least one active centre appear, because an empty dropdown would be a dead end.
- A centre with no worker still appears, with "0 workers", so the citizen knows nobody will review their readings yet before they choose it.
- The centre's own spelling of the state and district is saved, not whatever was typed.
- Submitting twice is refused (409), even if two submits race each other.
- Editing later (PATCH) merges the changes onto the saved profile and re-checks the whole thing, so an edit can never create a profile that sign-up would have refused.
- Saying no to health tracking is allowed, but logging vitals stays blocked until it is turned on in Settings.

Where the health centre list comes from

The `valid_phc_facilities` table is the list of real health centres (state, district, block, name). It is filled by `seeds/seed_phc_facilities.py`. Each centre also stores a **normalised name** (for example "P.H.C. Mylapore" and "Mylapore Primary Health Centre" both become "phc mylapore"), which `services/phc_registry.py` uses to match the centre a PHC worker typed at sign-up. If there is no single clear match, the worker is not linked, and an administrator must link them by hand. A worker with no centre sees nobody.

4. Security decisions, explained simply

Decision	Why
Passwords hashed with bcrypt	If the database leaks, the passwords cannot be read back.
Tokens stored only as SHA-256 hashes	Email links and refresh tokens are useless to someone who reads the database.
Refresh token in an httpOnly cookie, path /api/v1/auth	JavaScript cannot read it, and the browser only sends it to the auth endpoints.
Access token in memory, 15 minutes	Never in localStorage. A stolen token expires quickly.
Refresh tokens work once (rotation)	A copied token is caught the moment both copies are used, and the whole chain is signed out.
48-hour idle timeout	A forgotten session on a shared computer ends on its own.
Same message for wrong password and unknown email, same timing	Nobody can find out which emails have accounts.
Forgot password always gives the same answer	Same reason, and the 60-second limit stops inbox spam.
Reset password signs out every device	Throws out anyone who knew the old password.
Role read from the database on every request	Approvals and deactivations apply immediately, not after the token expires.
Fixed JWT algorithm and a 32+ character secret	Blocks known JWT attacks; the server will not start with a weak secret.
Onboarding answers in sessionStorage, cleared on sign-out	Health answers do not stay behind on a shared device.
Consent answers are append-only	There is a full history of who agreed to what, when, and from where.

5. Tests and settings

What	Where	Notes
Auth tests	`backend/tests/test_auth.py`	30 tests: sign-up, login, refresh and rotation, reuse detection, sessions, verification, password reset.
Onboarding tests	`backend/tests/test_onboarding.py`	17 tests: the registry lists, creating and editing the profile, facility checks.
Pattern test	`backend/tests/test_patterns.py`	3 tests: the employee ID pattern is the same in the frontend and backend.
Token lifetimes	`core/config.py`	ACCESS_TOKEN_EXPIRE_MINUTES = 15, REFRESH_TOKEN_EXPIRE_DAYS = 30.
Other timings	`core/security.py`	48h idle timeout, 10s reuse grace, 24h verification link, 1h reset link, 60s between emails.
Email	`backend/.env`	Set SMTP_HOST and related settings for real email; without them, emails print to the console.
Cookies	`backend/.env`	COOKIE_SECURE = true in production (HTTPS only).

Cheat sheet: explain each part in one line

Part	One-line explanation
Sign up	Creates the account with a hashed password, a pending application for workers, and emails a one-time verification link.
Verify email	The link's token is checked against its saved hash, works once, and expires after 24 hours.
Log in	Checks the password and starts a session: a 15-minute access token in memory and a 30-day refresh cookie.
Staying signed in	When the access token expires, the refresh cookie is swapped for a new pair, and each refresh token works only once.
Devices	Every login is a row in refresh_tokens; you can see them and end any of them.
Password reset	A 1-hour link, the same answer for every email, and all devices signed out afterwards.
Guards	The frontend sends people to the step they still owe; the backend refuses data to anyone without the right role.
Onboarding	5 steps saved in one transaction; the chosen health centre decides which workers see the citizen.
PHC approval	Workers wait as pending_phc until an administrator approves them (only the seed script does this today).